MercadinhoSys

Plano de Execução Técnica

Estado real do projeto · Março 2026 · Branch revisao/erp-analise

6	16	S-07	v1.0
sprints concluídas	sprints restantes	próxima sprint	meta de lançamento

Status do Projeto

Quadro de progresso baseado na análise do último diff. Atualizar ao início de cada sessão com o agente.

✓ Sprint 06 foi além do escopo planejado — fiado, ContaReceber, lock pessimista e MovimentacaoCaixa foram implementados antes do previsto. Isso antecipa parte das Sprints 08 e 10.

⚠ Débito técnico identificado: verify_logins_script.py commitado + print() de debug em vendas.py + fallback hardcoded estabelecimento_id. Resolver na Sprint 08 antes de avançar.

#	Sprint	Status	Observação
S-01	Banco de Dados — Schema Limpo	• concluída	*Colunas senha e is_super_admin confirmadas.*
S-02	Autenticação — Backend	• concluída	*is_super_admin lido do banco. JWT correto.*
S-03	Autenticação — Frontend	• concluída	*Login no browser funcionando.*
S-04	Gêmeo Digital — Injectors	• concluída	*set_password corrigido após session.add.*

S-05	Repositório — Limpeza	● **concluída**	*13 scripts removidos. .gitignore atualizado.*
S-06	PDV — Venda Completa	● **concluída**	*Fiado, caixa, rollback atômico implementados.*
S-07	**Seed Completa — Validação E2E**	▸ próxima	*Todos os módulos populados. Testes de isolamento.*
S-08	**Limpeza do Débito Técnico**	▸ próxima	*print(), fallback hardcoded, script não rastreado.*
S-09	Estoque Inteligente	○ pendente	
S-10	Caixa e Financeiro	○ pendente	
S-11	RH e Ponto	○ pendente	
S-12	Dashboard — KPIs Tempo Real	○ pendente	
S-13	Dashboard — ABC e RFM	○ pendente	
S-14	Alertas Proativos	○ pendente	
S-15	PWA — Mobile do Dono	○ pendente	*Pré-requisito: S-12*
S-16	PDV Mobile — Tablet	○ pendente	*Pré-requisito: S-09*
S-17	Onboarding Self-Service	○ pendente	
S-18	Billing — Pagar.me	○ pendente	*Pré-requisito: S-17*
S-19	White-Label	○ pendente	*Pré-requisito: S-18*
S-20	Segurança Final	○ pendente	*Pré-requisito: todas as features*
S-21	Performance e Docker Prod	○ pendente	*Pré-requisito: S-20*
S-22	Documentação e v1.0.0	○ pendente	*Sprint final de lançamento*

Débito Técnico — Resolver Antes de Avançar

Estes problemas foram introduzidos ou identificados na Sprint 06. São pequenos mas comprometem a qualidade em produção. A Sprint 08 é dedicada exclusivamente a eles.

Severidade	Problema	Arquivo	Ação
ALTO	Fallback hardcoded: estabelecimento_id = claims.get(...) or 1	vendas.py ~L.979	Retornar 401 se JWT não tiver estabelecimento_id
ALTO	print() de debug que vaza payload de venda nos logs	vendas.py ~L.985	Substituir por current_app.logger.debug()
MÉDIO	verify_logins_script.py commitado apesar do .gitignore	backend/verify_logins_script.py	git rm + commit
BAIXO	Busca de produto sem filtro de estabelecimento em código antigo	pdv.py (resolvido parcialmente)	Confirmar que nenhuma query retorna produto de outro tenant

Sprints Detalhadas — S-07 a S-14

Cada sprint foi escrita para ser copiada e enviada ao agente em uma sessão independente. Não enviar mais de uma sprint por sessão.

F·0 Agora — Consolidar o que foi construído

Seed completa e débito técnico eliminado. Só então avançar para novas features.

07

VALIDAÇÃO · FASE 0

Seed Completa — Gêmeo Digital Pronto para Demo

Após rodar a seed, o sistema tem dados reais em todos os módulos e está pronto para mostrar a qualquer cliente.

INFRA — Expandir seed_simulation_master.py para cobrir todos os módulos

`backend/seed_simulation_master.py`

Para cada um dos 5 estabelecimentos: mínimo 30 vendas distribuídas no mês com itens variados, 3 tipos de despesas, 3 funcionários com registros de ponto (entradas e saídas), 5 clientes com histórico de compra, pelo menos 2 pedidos de compra. A seed deve ser determinística — rodar duas vezes gera os mesmos dados.

TEST — Validar endpoints críticos após a seed

Terminal (script temporário, não commitar)

Testar com token de admin_elite: GET /api/dashboard/cientifico, GET /api/produtos, GET /api/vendas, GET /api/funcionarios, GET /api/despesas. Todos devem retornar 200 com pelo menos 1 registro. Anotar qualquer erro 500 para corrigir na Sprint 12.

TEST — Confirmar isolamento entre os 5 tenants

Terminal

Com token de admin_elite: listar produtos e guardar os IDs. Com token de admin_bom: listar produtos e confirmar que nenhum ID da lista anterior aparece. Repetir para GET /api/vendas.

CLEAN — Remover print() de debug deixados em vendas.py

backend/app/routes/vendas.py

Substituir print(f'Dados da venda recebidos: {data}') por current_app.logger.debug(). Substituir print(f'ERRO CRÍTICO VENDA: {e}') por current_app.logger.error(f'[VENDA] Erro crítico: {e}'). Esta é a única exceção ao 'não misturar débito com sprint de feature' — vale a pena resolver aqui porque é no mesmo arquivo.

CRITÉRIO DE ACEITE

- ☐ seed_simulation_master.py termina sem exceção
- ☐ Os 5 logins da seed retornam success: true
- ☐ GET /api/dashboard/cientifico retorna 200 com dados para admin_elite
- ☐ Produto do tenant A não aparece no GET /api/produtos do tenant B
- ☐ Zero print() restantes em backend/app/routes/vendas.py

COMMIT

```
git add backend/seed_simulation_master.py && git commit -m "feat(seed): seed completa com todos os módulos e validação E2E"
```

08

LIMPEZA · FASE 0

Débito Técnico — Código Pronto para Produção

Tudo que foi construído deve estar no nível de qualidade de produção antes de avançar.

CLEAN — Remover verify_logins_script.py do repositório

Terminal

git rm backend/verify_logins_script.py. Confirmar que git status não mostra o arquivo. Confirmar que o .gitignore tem a linha backend/verify_*.py. Se não tiver, adicionar.

FIX — Remover fallback hardcoded do estabelecimento_id

backend/app/routes/vendas.py ~L.979

A linha 'estabelecimento_id = int(claims.get("estabelecimento_id") or 1)' precisa ser: estabelecimento_id = claims.get('estabelecimento_id'). Se for None, retornar jsonify({'error': 'Token inválido'}), 401. Nunca usar tenant 1 como fallback.

CLEAN — Auditoria global de print() em todo o backend

`backend/app/routes/ (busca em todos os arquivos)`

Rodar: grep -rn 'print(' backend/app/routes/ | grep -v '.pyc'. Para cada ocorrência: se for log de info, usar current_app.logger.info(). Se for erro, usar current_app.logger.error(). Se for debug temporário, deletar. Zero print() no backend de produção.

FEAT — Adicionar nome do estabelecimento na resposta do POST /api/vendas

`backend/app/routes/vendas.py (bloco de resposta, ~L.1175)`

Incluir no JSON de resposta: estabelecimento_nome: estabelecimento.nome_fantasia. O frontend usa para montar o cabeçalho do recibo sem fazer uma segunda requisição.

CRITÉRIO DE ACEITE

- ☐ git log não mostra verify_logins_script.py
- ☐ grep -rn 'print(' backend/app/routes/ retorna zero resultados
- ☐ POST /api/vendas sem token retorna 401, não usa fallback do tenant 1
- ☐ Resposta do POST /api/vendas inclui o campo estabelecimento_nome

COMMIT

```
git add backend/ && git commit -m "fix(prod): remove debug, fallback perigoso e adiciona nome do estabelecimento no recibo"
```

F·2 Núcleo Operacional

O que o funcionário usa todo dia. Sem fricção, sem erro, sem precisar chamar suporte.

09

ESTOQUE · FASE 2

Estoque Inteligente — O Dono Sabe Antes de Acabar

Alertas automáticos de estoque baixo e vencimento. Movimentação rastreável com motivo.

AUDIT — CRUD completo de produto com validações de negócio

`backend/app/routes/produtos.py`

Criar produto valida: código de barras único por estabelecimento, quantidade_minima >= 0. Inativar não deleta — soft delete com ativo=False. PUT aceita atualização parcial (apenas os campos enviados).

FEAT — Endpoint GET /api/produtos/alertas

`backend/app/routes/produtos.py`

Retorna 3 arrays: criticos (quantidade <= 0), baixo_estoque (quantidade <= quantidade_minima e > 0), vencendo (data_validade nos próximos 30 dias). Cada item inclui id, nome, quantidade, quantidade_minima, data_validade. Resposta em < 100ms.

FEAT — POST /api/produtos/{id}/movimentar — rastreabilidade

`backend/app/routes/produtos.py`

Body: { tipo: 'entrada'|'saida'|'ajuste', quantidade: float, motivo: string (min 10 chars) }. Cria MovimentacaoEstoque com tipo='ajuste_manual' e funcionario_id do JWT. Retorna o produto atualizado.

FEAT — GET /api/produtos/barcode/{codigo} — otimizado para PDV

`backend/app/routes/produtos.py`

Busca produto por código de barras do estabelecimento do JWT. Sem paginação. Sem joins desnecessários. Deve responder em < 50ms — será chamado a cada leitura de câmera no PDV Mobile.

CRITÉRIO DE ACEITE

- ☐ GET /api/produtos/alertas retorna as 3 arrays com dados da seed
- ☐ POST /api/produtos/{id}/movimentar sem motivo retorna 400
- ☐ GET /api/produtos/barcode/{codigo} responde em menos de 100ms
- ☐ Produto inativado não aparece no GET /api/produtos padrão

COMMIT

```
git add backend/app/routes/produtos.py && git commit -m "feat(estoque): alertas, movimentação rastreável e busca por barcode"
```

10

FINANCEIRO · FASE 2

Caixa — Fechamento que Bate com a Gaveta

Sangria, suprimento, fechamento auditável. O gerente sabe exatamente onde está cada centavo.

AUDIT — Abertura de caixa idempotente

`backend/app/routes/ (caixa.py ou financeiro.py)`

POST /api/caixa/abrir: verificar se já existe caixa com status='aberto' para o estabelecimento. Se sim, retornar 409 com mensagem: 'Já existe um caixa aberto. Feche o caixa atual antes de abrir um novo.' Registrar saldo_inicial, funcionario_id, hora_abertura.

AUDIT — Fechamento com cálculo auditável

`backend/app/routes/ (caixa.py ou financeiro.py)`

POST /api/caixa/fechar: calcular saldo_calculado = saldo_inicial + SUM(entradas dinheiro) - SUM(sangrias). Salvar saldo_informado (enviado pelo operador) e diferenca = saldo_informado - saldo_calculado. Registrar hora_fechamento e funcionario_id de quem fechou.

FEAT — Sangria e suprimento durante o turno

`backend/app/routes/ (caixa.py ou financeiro.py)`

POST /api/caixa/sangria: { valor, motivo (obrigatório) }. Cria MovimentacaoCaixa tipo='sangria', diminui saldo_atual. POST /api/caixa/suprimento: { valor, motivo }. Cria MovimentacaoCaixa tipo='suprimento', aumenta saldo_atual.

FEAT — GET /api/caixa/{id}/resumo — para o relatório de fechamento

`backend/app/routes/ (caixa.py ou financeiro.py)`

Retorna: { total_por_forma_pagamento: {dinheiro, cartao, pix, fiado}, qtd_vendas, total_sangrias, total_suprimentos, saldo_inicial, saldo_calculado, saldo_informado, diferenca, status, ultimas_movimentacoes: [] }.

CRITÉRIO DE ACEITE

- ☐ Abrir segundo caixa retorna 409 com mensagem clara
- ☐ Saldo calculado no fechamento é consistente com as movimentações
- ☐ Sangria aparece no GET /api/caixa/{id}/resumo com valor correto
- ☐ Campo diferenca é calculado e salvo corretamente

COMMIT

```
git add backend/app/routes/ && git commit -m "feat(caixa): sangria, suprimento,
fechamento auditável e resumo de turno"
```

11

RH · FASE 2

Ponto e RH — Gestão de Equipe com Dados Reais

O gerente sabe quem chegou, quem atrasou e quanto a folha custou este mês.

AUDIT — Registro de ponto com validação de duplicidade

`backend/app/routes/ (ponto.py ou rh.py)`

POST /api/ponto/registrar: verificar se já existe registro do mesmo tipo (entrada/saida) no mesmo dia para o funcionario_id. Se sim, retornar 409. Calcular minutos_atraso comparando com ConfiguracaoHorario.hora_entrada. Aceitar lat/lng opcional para validação de presença.

FEAT — Espelho de ponto mensal — GET /api/ponto/espelho

`backend/app/routes/ (ponto.py ou rh.py)`

Query params: funcionario_id, mes (formato YYYY-MM). Retorna array de dias: { data, hora_entrada, hora_saida, minutos_trabalhados, minutos_atraso, status: 'normal'|'atrasado'|'falta'|'folga' }. Inclui totais do mês: dias_trabalhados, total_atrasos_min, total_horas.

FEAT — GET /api/rh/resumo — custo e pontualidade do mês

`backend/app/routes/ (ponto.py ou rh.py)`

Query param: mes. Retorna: { total_salarios, total_beneficios, custo_total_rh, taxa_pontualidade_pct, funcionarios_com_atraso, media_minutos_atraso, total_horas_extras }. Dados já existem no data_layer — este endpoint só agrega e serializa.

CLEAN — Remover print() de debug do data_layer.py

`backend/app/dashboard_cientifico/data_layer.py`

Rodar: grep -n 'print(f"DEBUG' backend/app/dashboard_cientifico/data_layer.py. Para cada linha encontrada: deletar. São logs de desenvolvimento que poluem produção e expõem estrutura interna da query.

CRITÉRIO DE ACEITE

- ☐ Registrar entrada duas vezes no mesmo dia retorna 409
- ☐ GET /api/ponto/espelho retorna array com dias corretos do mês
- ☐ GET /api/rh/resumo retorna taxa_pontualidade calculada
- ☐ Zero print(f'DEBUG' em data_layer.py

COMMIT

```
git add backend/app/routes/ backend/app/dashboard_cientifico/ && git commit -m
"feat(rh): espelho de ponto, resumo mensal e limpeza de debug"
```

F·3 Inteligência do Dono

O diferencial competitivo real. Nenhum sistema no segmento entrega isso.

1

2

DASHBOARD · FASE 3

KPIs em Tempo Real — O Dono Vê o Negócio em 10 Segundos

Ao logar, o dono sabe se hoje está sendo um bom dia — sem precisar perguntar nada a ninguém.

FIX — Corrigir erros 500 no GET /api/dashboard/cientifico

`backend/app/dashboard_cientifico/`

Rodar com token de admin_elite após a seed. Coletar traceback do Docker log. Corrigir uma função por vez — não tentar corrigir tudo simultaneamente. Prioridade: get_resumo_financeiro > get_top_produtos > demais funções.

FEAT — Endpoint GET /api/dashboard/resumo-dia

`backend/app/routes/dashboard.py`

Retorna: { faturamento_hoje, faturamento_ontem, delta_pct, qtd_vendas_hoje, ticket_medio_hoje, margem_estimada_pct, caixa_status: 'aberto'|'fechado'|'nao_aberto' }. Dados em tempo real. Cache máximo de 60 segundos via Redis.

FEAT — Cards de KPI na tela inicial do dashboard

`frontend/src/features/dashboard/DashboardPage.tsx`

4 cards após o login: Faturamento Hoje (com seta delta % vs ontem), Vendas Hoje (contagem), Ticket Médio, Status do Caixa. Dados do GET /api/dashboard/resumo-dia. Loading skeleton enquanto carrega — sem flash de tela em branco.

FEAT — Gráfico de vendas por hora do dia

`frontend/src/features/dashboard/`

Gráfico de barras: 24 horas no eixo X, faturamento no eixo Y. Hora atual destacada. Mostra o pico de movimento. Usa GET /api/dashboard/cientifico que já retorna get_vendas_por_hora(). Sem nova rota necessária.

CRITÉRIO DE ACEITE

- ☐ GET /api/dashboard/cientifico retorna 200 sem erro 500
- ☐ GET /api/dashboard/resumo-dia responde em menos de 200ms
- ☐ 4 cards de KPI aparecem na tela após login
- ☐ Gráfico por hora exibe barras com dados reais da seed

COMMIT

```
git add backend/ frontend/ && git commit -m "feat(dashboard): KPIs tempo real,
resumo-dia e gráfico por hora"
```

13

DASHBOARD · FASE 3

Análise ABC e RFM — Onde Está o Dinheiro de Verdade

O dono descobre que 3 produtos geram 60% do lucro. E para de comprar os 40 que não vendem.

FEAT — Tela de Análise ABC de Produtos

`frontend/src/features/dashboard/`

Tabela: Produto | Faturamento | % do Total | Margem | Classe. Classe A em verde, B em âmbar, C em cinza. Ordenada por faturamento decrescente. Dados de GET /api/relatorios/rentabilidade-abc que já existe. Botão 'Exportar CSV' que baixa a lista.

FEAT — Painel RFM de Clientes com ações sugeridas

`frontend/src/features/dashboard/`

3 seções: Campeões (R alto, F alto, M alto), Em Risco (compravam bem, pararam há 30+ dias), Dormentes (sem compra há 60+ dias). Cada seção mostra contagem e amostra de 3 clientes. Botão 'Exportar contatos' baixa CSV com nome e celular. Dados de GET /api/relatorios/rfm.

FEAT — Widget de margem por produto — alertas de prejuízo

`frontend/src/features/dashboard/`

Top 10 produtos com menor margem. Margem calculada como (preco_venda - custo_unitario) / preco_venda × 100. Linha vermelha para margem < 0, amarela para < 5%. Tooltip ao passar o mouse mostra: custo, preço, margem exata.

FEAT — Comparativo semanal — setas de crescimento ou queda

`frontend/src/features/dashboard/`

No card de Faturamento Hoje: seta verde se faturamento_semana_atual > faturamento_semana_anterior, vermelha se menor. Percentual de diferença ao lado. Sem nova rota — calcular no frontend com os dados já retornados pelo resumo-dia.

CRITÉRIO DE ACEITE

- ☐ Tabela ABC carrega e classifica produtos em A, B e C corretamente
- ☐ Painel RFM mostra pelo menos 1 cliente em cada categoria após a seed
- ☐ Widget de margem destaca produtos com margem negativa em vermelho
- ☐ Seta de comparativo aponta para direção correta com % correto

COMMIT

```
git add frontend/ && git commit -m "feat(dashboard): análise ABC, painel RFM com
export e comparativo semanal"
```

14

ALERTAS · FASE 3

Alertas Proativos — O Sistema que Pensa pelo Dono

O sistema detecta problemas antes do dono perceber. Sem que ele precise pedir.

FEAT — Endpoint GET /api/alertas/ativos

`backend/app/routes/alertas.py (novo arquivo)`

Agrega alertas de 4 fontes: (1) produtos críticos do GET /api/produtos/alertas, (2) caixa não aberto após as 8h, (3) ContaReceber vencendo nos próximos 7 dias, (4) anomalia de vendas (faturamento atual < 60% da média histórica no mesmo horário). Retorna lista ordenada por severidade: crítico > atenção > informativo.

FEAT — Ícone de sino no header com badge de contagem

`frontend/src/components/layout/HeaderProfessional.tsx`

Ícone de sino à direita do header. Badge vermelho com contagem de alertas críticos não lidos. Ao clicar: dropdown com lista de alertas. Cada alerta tem: ícone de severidade, mensagem, timestamp e botão 'Dispensar'.

FEAT — Hook useAlertas — polling a cada 5 minutos

`frontend/src/hooks/useAlertas.ts (novo arquivo)`

Hook React que chama GET /api/alertas/ativos ao montar o componente e a cada 300 segundos. Retorna: { alertas, contagem_criticos, loading, refetch }. Se aparecer alerta com severidade 'crítico', disparar toast automático com a mensagem.

FEAT — Alerta de anomalia de vendas — detecção simples

`backend/app/routes/alertas.py`

Calcular: media_historica = média dos últimos 14 dias no mesmo intervalo de hora. Se faturamento_atual < media_historica × 0.6, gerar alerta. Mensagem: 'Movimento 40% abaixo da média para este horário.' Sem ML — média aritmética simples.

CRITÉRIO DE ACEITE

- ☐ GET /api/alertas/ativos retorna pelo menos 1 alerta após a seed
- ☐ Badge do sino mostra contagem correta de alertas críticos
- ☐ Toast automático aparece quando chega alerta crítico novo
- ☐ Dispensar um alerta remove ele da lista sem recarregar a página

COMMIT

```
git add backend/ frontend/ && git commit -m "feat(alertas): central de alertas, polling, anomalia de vendas e sino no header"
```

Protocolo de Trabalho com o Gemini Flash

Copie o bloco abaixo e envie ao início de cada sessão com o agente. Sem estas instruções, o comportamento padrão do modelo vai gerar problemas.

> **i** Instrução padrão para copiar e colar no início de cada sessão com o Gemini:

```
Branch de trabalho: revisao/erp-analise. Nunca commitar na master.
Escopo: apenas a Sprint [X] deste documento. Nada fora do escopo.
Logging: usar current_app.logger. Nunca usar print() no backend.
Debug: qualquer arquivo de teste criado deve ser deletado antes do commit.
Erro: se não resolver em 2 tentativas, parar e mostrar o traceback completo.
Critério: não avançar para a próxima tarefa sem confirmar o aceite da atual.
```

Regra	Por quê falha sem ela
Uma sprint por sessão	O modelo perde contexto e começa a misturar tarefas de sprints diferentes
Enviar só os arquivos da sprint	Contexto menor = menos alucinação. O modelo não deve inferir conteúdo de arquivo
Confirmar aceite antes de avançar	O modelo assume que a tarefa passou quando não foi testada
Proibir print() explicitamente	O padrão do modelo é usar print() para debug — já aconteceu na Sprint 06
Proibir commits de scripts de debug	O verify_logins_script.py foi commitado porque não havia instrução contrária
2 tentativas máximas por erro	Evita que o modelo gaste 30 minutos tentando uma solução errada em loop